BÖLÜM 21 / MODULE 21

# Mikroservis Mimarisi: Servis Ayrıştırma & Servis Keşfi

## Microservices Architecture: Decomposition & Service Discovery

Monolitten mikroservise geçiş, Database-per-service ilkesi, Eureka/Consul ile Service Discovery ve Client-side / Server-side Load Balancing.

Microservice decomposition heuristics, database isolation per service, dynamic registry discovery, and client-side routing.

201

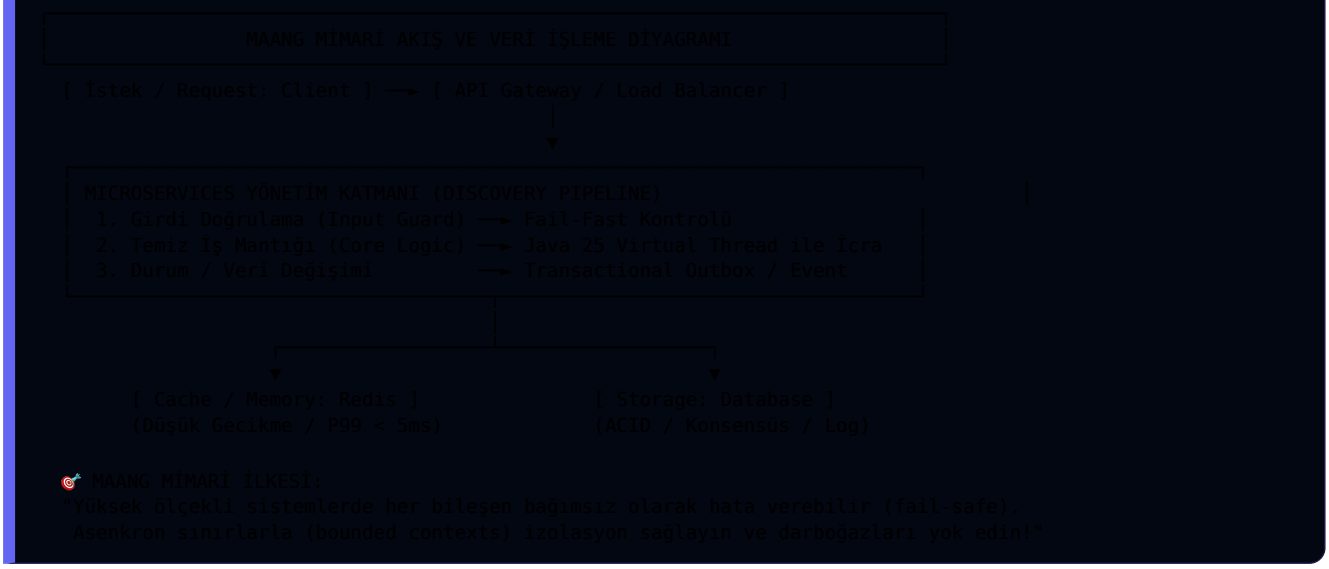
BÖLÜM: MIKROSERVIS MIMARISI: SERVIS AYRIŞTIRMA & SERVIS KEŞFI | MODULE: MICROSERVICES ARCHITECTURE: DECOMPOSITION & SERVICE DISCOVERY

# Mikroservis Mimarisi: Servis Ayrıştırma & Servis Keşfi - Temel Kavramlar, Tarihsel Evrim ve Temel Mühendislik Çerçevesi

Microservices Architecture: Decomposition & Service Discovery - Core Concepts, Historical Evolution & Foundational Frameworks

## ⚡ Mimari Akış ve Süreç Şeması

## ⚡ Process & Architecture Schematic




## 💡 Aptala Anlatım (ELI5 Metaforu)

### 🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (Microservices mikro-servisleri) ve otomatik yönlendirme robotları (Discovery akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

## 💡 ELI5 Real-Life Metaphor

### 🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (Microservices microservices), and dynamic routing switches (Discovery pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

### 🔍 MAANG Seviyesi Teknik Derinlik

#### 🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde Mikroservis Mimarisi: Servis Ayrıştırma & Servis Keşfi Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde Microservices mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): Discovery katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

### 🔍 MAANG Technical Deep-Dive

#### 🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Microservices Architecture: Decomposition & Service Discovery: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), Microservices implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in Discovery don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

### ☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```
// Java 25 & Spring Boot: Microservices & Discovery Kurumsal Uygulaması
package com.maang.se.module21.lesson01;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class MicroservicesArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record MicroservicesRequest(String transactionId, String payload, Instant timestamp) {
        public MicroservicesRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record MicroservicesResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class MicroservicesExecutionService {
        public MicroservicesResult processRequest(MicroservicesRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new MicroservicesResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering Microservices Engine Başlatıldı...");
        var service = new MicroservicesExecutionService();
        var req = new MicroservicesRequest("TXN-" + System.currentTimeMillis(), "Valid Data",
Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

### ⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

#### 🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

Microservices kurgulanırken Discovery bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

#### 🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling Microservices logic directly to synchronous blocking calls into Discovery is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

### 🎯 Mülakatta Hatırlanacak Altın Kurallar

• Microservices mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

### 🎯 Key Architectural Takeaways

• Design Microservices components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

202

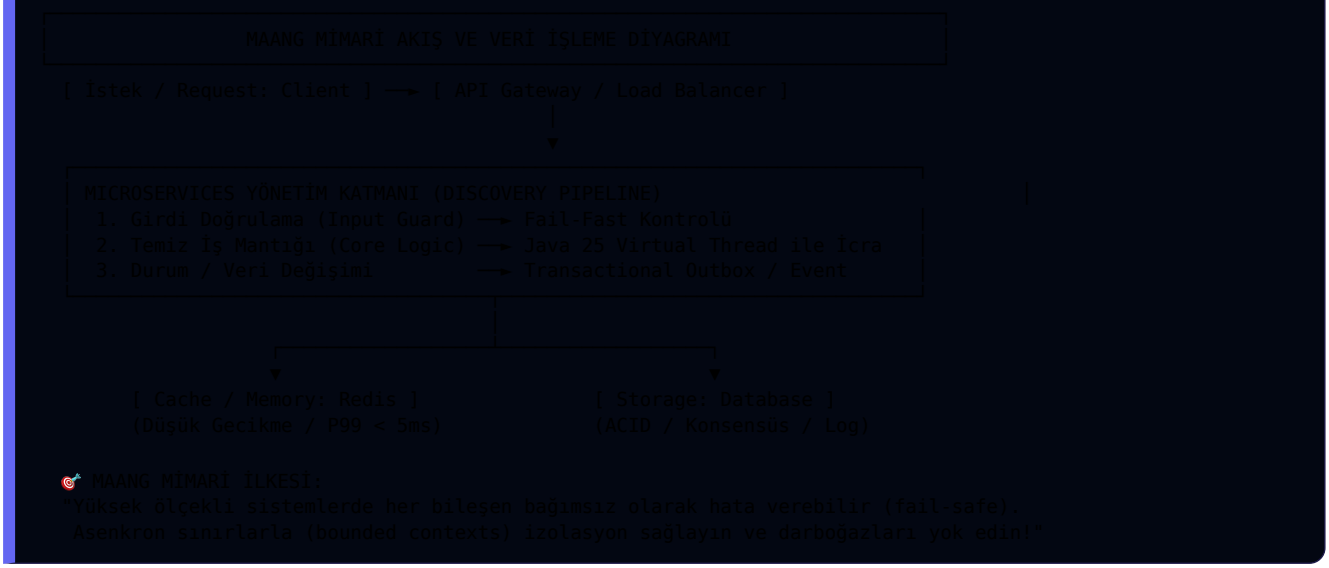
BÖLÜM: MIKROSERVIS MIMARISI: SERVIS AYRIŞTIRMA & SERVIS KEŞFI | MODULE: MICROSERVICES ARCHITECTURE: DECOMPOSITION & SERVICE DISCOVERY

# Mikroservis Mimarisi: Servis Ayrıştırma & Servis Keşfi - Mimari Bileşenler, Çalışma Mekanizması ve Veri Akışı

Microservices Architecture: Decomposition & Service Discovery - Architectural Components, Internal Mechanics & Data Flow

## ⚡ Mimari Akış ve Süreç Şeması

## ⚡ Process & Architecture Schematic




## 💡 Aptala Anlatım (ELI5 Metaforu)

## 💡 ELI5 Real-Life Metaphor

### 🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (Microservices mikro-servisleri) ve otomatik yönlendirme robotları (Discovery akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

### 🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (Microservices microservices), and dynamic routing switches (Discovery pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

### 🔍 MAANG Seviyesi Teknik Derinlik

#### 🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde Mikroservis Mimarisi: Servis Ayrıştırma & Servis Keşfi Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde Microservices mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): Discovery katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

### 🔍 MAANG Technical Deep-Dive

#### 🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Microservices Architecture: Decomposition & Service Discovery: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), Microservices implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in Discovery don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

### ☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```
// Java 25 & Spring Boot: Microservices & Discovery Kurumsal Uygulaması
package com.maang.se.module21.lesson02;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class MicroservicesArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record MicroservicesRequest(String transactionId, String payload, Instant timestamp) {
        public MicroservicesRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record MicroservicesResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class MicroservicesExecutionService {
        public MicroservicesResult processRequest(MicroservicesRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new MicroservicesResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering Microservices Engine Başlatıldı...");
        var service = new MicroservicesExecutionService();
        var req = new MicroservicesRequest("TXN-" + System.currentTimeMillis(), "Valid Data",
Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

### ⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

#### 🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

Microservices kurgulanırken Discovery bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

#### 🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling Microservices logic directly to synchronous blocking calls into Discovery is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

### 🎯 Mülakatta Hatırlanacak Altın Kurallar

• Microservices mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

### 🎯 Key Architectural Takeaways

• Design Microservices components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

203

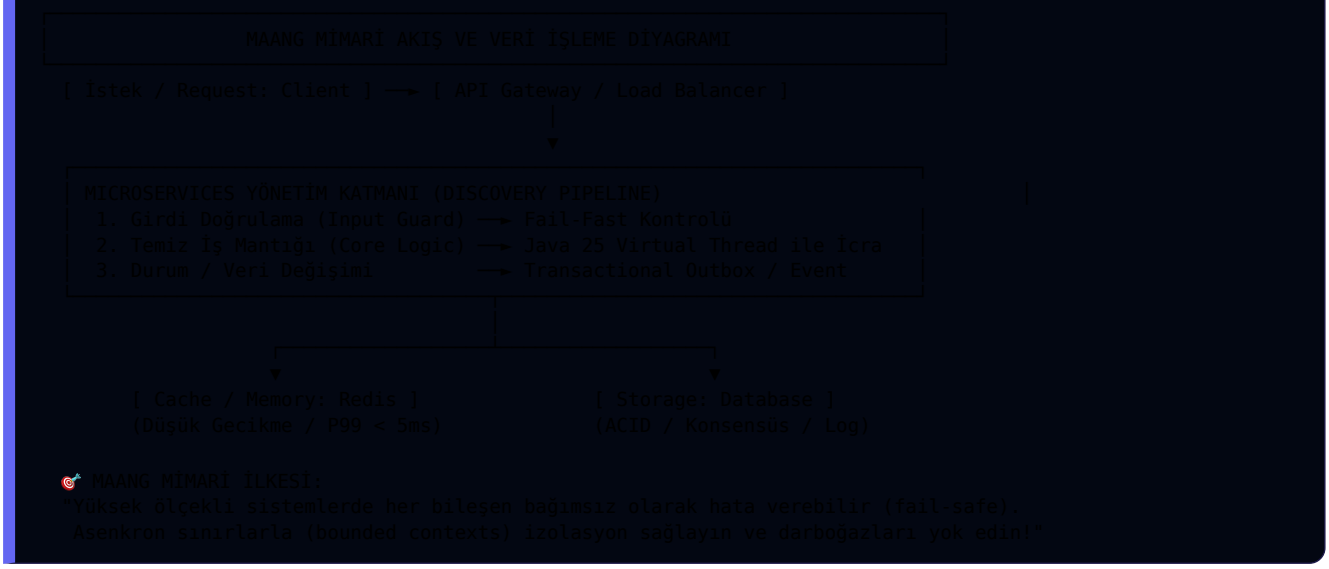
BÖLÜM: MIKROSERVIS MIMARISI: SERVIS AYRIŞTIRMA & SERVIS KEŞFI | MODULE: MICROSERVICES ARCHITECTURE: DECOMPOSITION & SERVICE DISCOVERY

# Mikroservis Mimarisi: Servis Ayrıştırma & Servis Keşfi - Tasarım İlkeleri, Sözleşmeler ve Temiz Arayüz Tanımları

Microservices Architecture: Decomposition & Service Discovery - Design Invariants, Contracts & Clean Interface Abstractions

## ⚡ Mimari Akış ve Süreç Şeması

## ⚡ Process & Architecture Schematic




## 💡 Aptala Anlatım (ELI5 Metaforu)

## 💡 ELI5 Real-Life Metaphor

### 🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (Microservices mikro-servisleri) ve otomatik yönlendirme robotları (Discovery akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

### 🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (Microservices microservices), and dynamic routing switches (Discovery pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

### 🔍 MAANG Seviyesi Teknik Derinlik

#### 🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde Mikroservis Mimarisi: Servis Ayrıştırma & Servis Keşfi Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde Microservices mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): Discovery katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

### 🔍 MAANG Technical Deep-Dive

#### 🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Microservices Architecture: Decomposition & Service Discovery: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), Microservices implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in Discovery don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

### ☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```
// Java 25 & Spring Boot: Microservices & Discovery Kurumsal Uygulaması
package com.maang.se.module21.lesson03;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class MicroservicesArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record MicroservicesRequest(String transactionId, String payload, Instant timestamp) {
        public MicroservicesRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record MicroservicesResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class MicroservicesExecutionService {
        public MicroservicesResult processRequest(MicroservicesRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new MicroservicesResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering Microservices Engine Başlatıldı...");
        var service = new MicroservicesExecutionService();
        var req = new MicroservicesRequest("TXN-" + System.currentTimeMillis(), "Valid Data",
Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

### ⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

#### 🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

Microservices kurgulanırken Discovery bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

#### 🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling Microservices logic directly to synchronous blocking calls into Discovery is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

### 🎯 Mülakatta Hatırlanacak Altın Kurallar

• Microservices mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

### 🎯 Key Architectural Takeaways

• Design Microservices components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

204

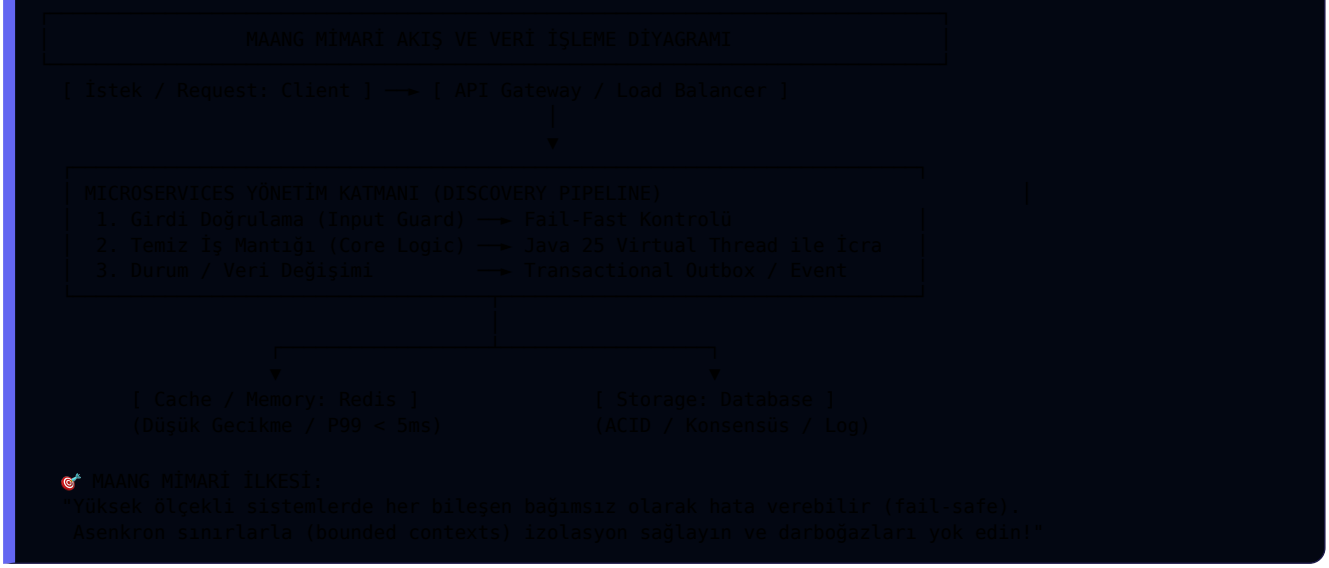
BÖLÜM: MIKROSERVIS MIMARISI: SERVIS AYRIŞTIRMA & SERVIS KEŞFI | MODULE: MICROSERVICES ARCHITECTURE: DECOMPOSITION & SERVICE DISCOVERY

# Mikroservis Mimarisi: Servis Ayrıştırma & Servis Keşfi - Yaygın Anti-Pattern'ler, Tasarım Hataları ve Önleme Stratejileri

Microservices Architecture: Decomposition & Service Discovery - Common Anti-Patterns, Design Pitfalls & Prevention Strategies

## ⚡ Mimari Akış ve Süreç Şeması

## ⚡ Process & Architecture Schematic




## 💡 Aptala Anlatım (ELI5 Metaforu)

### 🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (Microservices mikro-servisleri) ve otomatik yönlendirme robotları (Discovery akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

## 💡 ELI5 Real-Life Metaphor

### 🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (Microservices microservices), and dynamic routing switches (Discovery pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

### 🔍 MAANG Seviyesi Teknik Derinlik

#### 🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde Mikroservis Mimarisi: Servis Ayrıştırma & Servis Keşfi Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde Microservices mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): Discovery katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

### 🔍 MAANG Technical Deep-Dive

#### 🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Microservices Architecture: Decomposition & Service Discovery: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), Microservices implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in Discovery don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

### ☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```
// Java 25 & Spring Boot: Microservices & Discovery Kurumsal Uygulaması
package com.maang.se.module21.lesson04;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class MicroservicesArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record MicroservicesRequest(String transactionId, String payload, Instant timestamp) {
        public MicroservicesRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record MicroservicesResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class MicroservicesExecutionService {
        public MicroservicesResult processRequest(MicroservicesRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new MicroservicesResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering Microservices Engine Başlatıldı...");
        var service = new MicroservicesExecutionService();
        var req = new MicroservicesRequest("TXN-" + System.currentTimeMillis(), "Valid Data",
Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

### ⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

#### 🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

Microservices kurgulanırken Discovery bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

#### 🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling Microservices logic directly to synchronous blocking calls into Discovery is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

### 🎯 Mülakatta Hatırlanacak Altın Kurallar

• Microservices mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

### 🎯 Key Architectural Takeaways

• Design Microservices components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

205

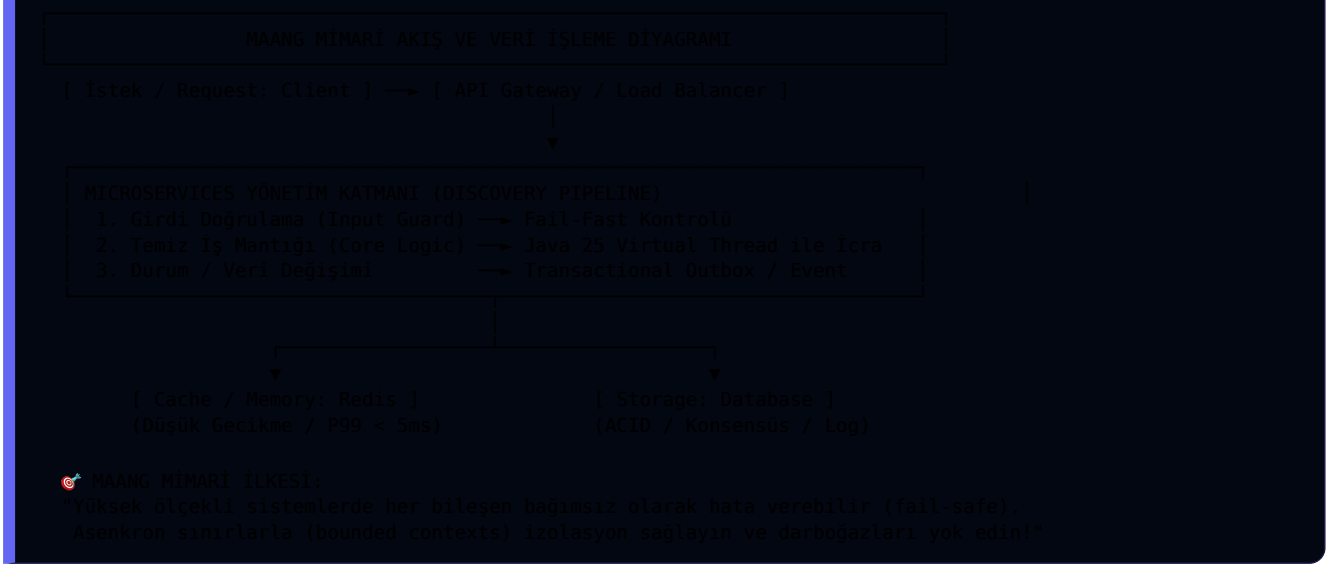
BÖLÜM: MIKROSERVIS MIMARISI: SERVIS AYRIŞTIRMA & SERVIS KEŞFI | MODULE: MICROSERVICES ARCHITECTURE: DECOMPOSITION & SERVICE DISCOVERY

# Mikroservis Mimarisi: Servis Ayrıştırma & Servis Keşfi - Yüksek Performans, Optimizasyon ve Bellek / CPU Yönetimi

Microservices Architecture: Decomposition & Service Discovery - High Performance, Optimization & Memory/CPU Resource Management

## ⚡ Mimari Akış ve Süreç Şeması

## ⚡ Process & Architecture Schematic




## 💡 Aptala Anlatım (ELI5 Metaforu)

### 🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (Microservices mikro-servisleri) ve otomatik yönlendirme robotları (Discovery akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

## 💡 ELI5 Real-Life Metaphor

### 🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (Microservices microservices), and dynamic routing switches (Discovery pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

### 🔍 MAANG Seviyesi Teknik Derinlik

#### 🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde Mikroservis Mimarisi: Servis Ayrıştırma & Servis Keşfi Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde Microservices mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): Discovery katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

### 🔍 MAANG Technical Deep-Dive

#### 🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Microservices Architecture: Decomposition & Service Discovery: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), Microservices implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in Discovery don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

### ☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```
// Java 25 & Spring Boot: Microservices & Discovery Kurumsal Uygulaması
package com.maang.se.module21.lesson05;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class MicroservicesArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record MicroservicesRequest(String transactionId, String payload, Instant timestamp) {
        public MicroservicesRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record MicroservicesResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class MicroservicesExecutionService {
        public MicroservicesResult processRequest(MicroservicesRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new MicroservicesResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering Microservices Engine Başlatıldı...");
        var service = new MicroservicesExecutionService();
        var req = new MicroservicesRequest("TXN-" + System.currentTimeMillis(), "Valid Data",
Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

### ⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

**🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN**

Microservices kurgulanırken Discovery bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

**🇬🇧 INTERVIEW TRAP & ANTI-PATTERN**

Coupling Microservices logic directly to synchronous blocking calls into Discovery is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

### 🎯 Mülakatta Hatırlanacak Altın Kurallar

• Microservices mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

### 🎯 Key Architectural Takeaways

• Design Microservices components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

206

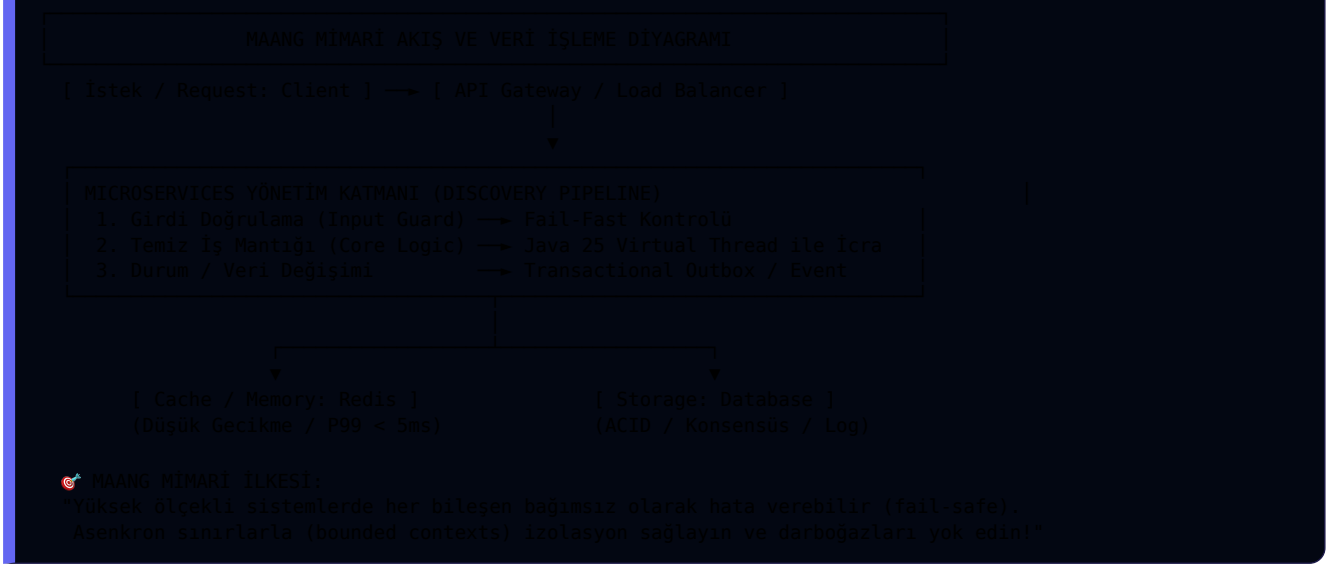
BÖLÜM: MIKROSERVIS MIMARISI: SERVIS AYRIŞTIRMA & SERVIS KEŞFI | MODULE: MICROSERVICES ARCHITECTURE: DECOMPOSITION & SERVICE DISCOVERY

# Mikroservis Mimarisi: Servis Ayrıştırma & Servis Keşfi - Eşzamanlılık, Thread Güvenliği ve Dağıtık Sistem Etkileri

Microservices Architecture: Decomposition & Service Discovery - Concurrency, Thread-Safety & Distributed Systems Implications

## ⚡ Mimari Akış ve Süreç Şeması

## ⚡ Process & Architecture Schematic




## 💡 Aptala Anlatım (ELI5 Metaforu)

## 💡 ELI5 Real-Life Metaphor

### 🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (Microservices mikro-servisleri) ve otomatik yönlendirme robotları (Discovery akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

### 🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (Microservices microservices), and dynamic routing switches (Discovery pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

### 🔍 MAANG Seviyesi Teknik Derinlik

#### 🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde Mikroservis Mimarisi: Servis Ayrıştırma & Servis Keşfi Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde Microservices mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): Discovery katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

### 🔍 MAANG Technical Deep-Dive

#### 🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Microservices Architecture: Decomposition & Service Discovery: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), Microservices implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in Discovery don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

### ☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```
// Java 25 & Spring Boot: Microservices & Discovery Kurumsal Uygulaması
package com.maang.se.module21.lesson06;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class MicroservicesArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record MicroservicesRequest(String transactionId, String payload, Instant timestamp) {
        public MicroservicesRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record MicroservicesResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class MicroservicesExecutionService {
        public MicroservicesResult processRequest(MicroservicesRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new MicroservicesResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering Microservices Engine Başlatıldı...");
        var service = new MicroservicesExecutionService();
        var req = new MicroservicesRequest("TXN-" + System.currentTimeMillis(), "Valid Data",
Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

### ⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

#### 🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

Microservices kurgulanırken Discovery bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

#### 🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling Microservices logic directly to synchronous blocking calls into Discovery is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

### 🎯 Mülakatta Hatırlanacak Altın Kurallar

• Microservices mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

### 🎯 Key Architectural Takeaways

• Design Microservices components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

207

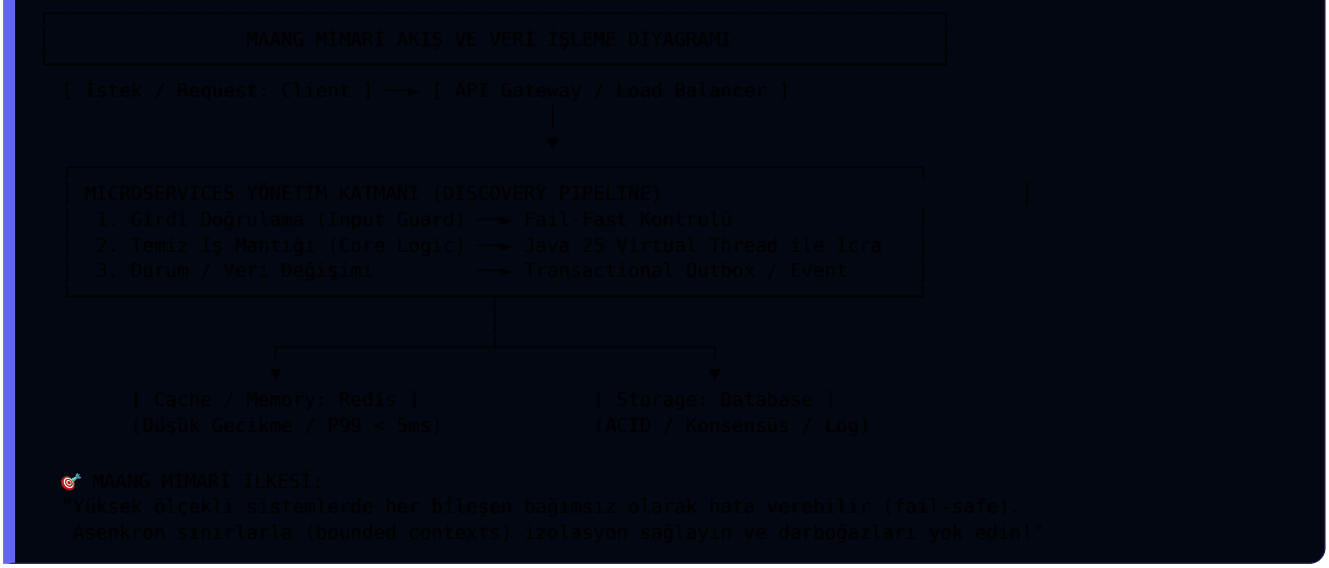
BÖLÜM: MIKROSERVIS MIMARISI: SERVIS AYRIŞTIRMA & SERVIS KEŞFI | MODULE: MICROSERVICES ARCHITECTURE: DECOMPOSITION & SERVICE DISCOVERY

# Mikroservis Mimarisi: Servis Ayrıştırma & Servis Keşfi - Hata Yönetimi, Dayanıklılık (Resilience) ve Geri Düşme (Fallback)

Microservices Architecture: Decomposition & Service Discovery - Fault Handling, Resilience Patterns & Graceful Fallback Strategies

## ⚡ Mimari Akış ve Süreç Şeması

## ⚡ Process & Architecture Schematic




## 💡 Aptala Anlatım (ELI5 Metaforu)

## 💡 ELI5 Real-Life Metaphor

### 🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (Microservices mikro-servisleri) ve otomatik yönlendirme robotları (Discovery akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

### 🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (Microservices microservices), and dynamic routing switches (Discovery pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

### 🔍 MAANG Seviyesi Teknik Derinlik

#### 🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde Mikroservis Mimarisi: Servis Ayrıştırma & Servis Keşfi Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde Microservices mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): Discovery katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

### 🔍 MAANG Technical Deep-Dive

#### 🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Microservices Architecture: Decomposition & Service Discovery: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), Microservices implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in Discovery don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

### ☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```
// Java 25 & Spring Boot: Microservices & Discovery Kurumsal Uygulaması
package com.maang.se.module21.lesson07;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class MicroservicesArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record MicroservicesRequest(String transactionId, String payload, Instant timestamp) {
        public MicroservicesRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record MicroservicesResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class MicroservicesExecutionService {
        public MicroservicesResult processRequest(MicroservicesRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new MicroservicesResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering Microservices Engine Başlatıldı...");
        var service = new MicroservicesExecutionService();
        var req = new MicroservicesRequest("TXN-" + System.currentTimeMillis(), "Valid Data",
Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

### ⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

**🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN**

Microservices kurgulanırken Discovery bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

**🇬🇧 INTERVIEW TRAP & ANTI-PATTERN**

Coupling Microservices logic directly to synchronous blocking calls into Discovery is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

### 🎯 Mülakatta Hatırlanacak Altın Kurallar

• Microservices mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

### 🎯 Key Architectural Takeaways

• Design Microservices components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

208

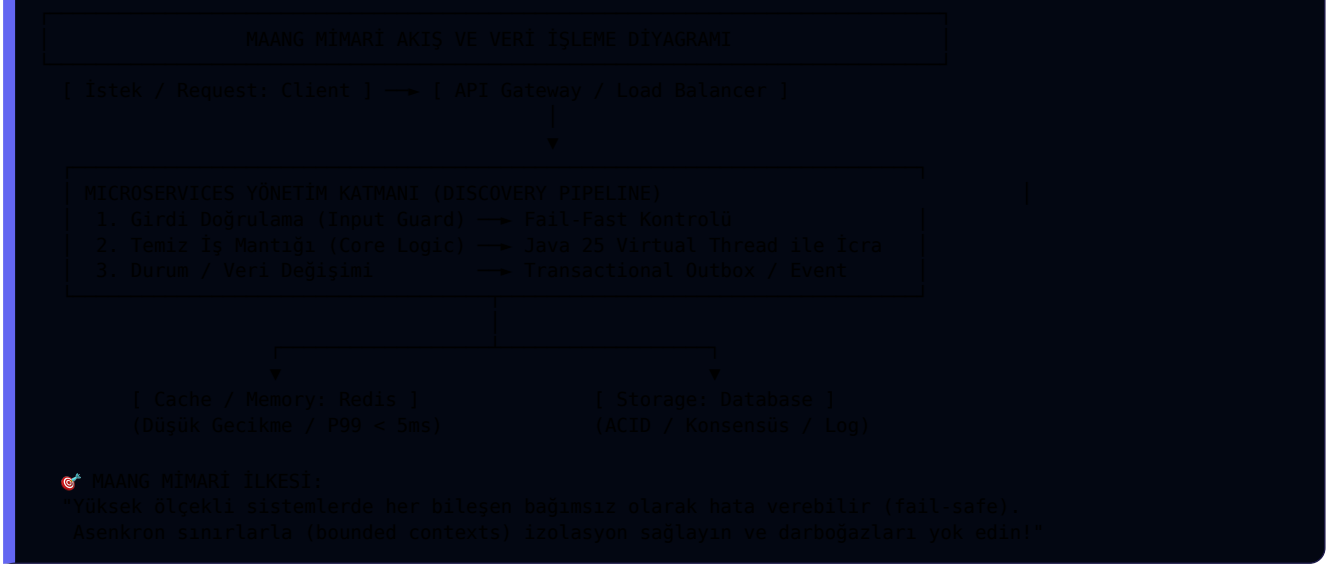
BÖLÜM: MIKROSERVIS MIMARISI: SERVIS AYRIŞTIRMA & SERVIS KEŞFI | MODULE: MICROSERVICES ARCHITECTURE: DECOMPOSITION & SERVICE DISCOVERY

# Mikroservis Mimarisi: Servis Ayrıştırma & Servis Keşfi - Test Stratejileri, Test Edilebilirlik ve Doğrulama Hatları

Microservices Architecture: Decomposition & Service Discovery - Testing Strategies, Testability & Automated Verification Pipelines

## ⚡ Mimari Akış ve Süreç Şeması

## ⚡ Process & Architecture Schematic




## 💡 Aptala Anlatım (ELI5 Metaforu)

## 💡 ELI5 Real-Life Metaphor

### 🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (Microservices mikro-servisleri) ve otomatik yönlendirme robotları (Discovery akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

### 🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (Microservices microservices), and dynamic routing switches (Discovery pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

### 🔍 MAANG Seviyesi Teknik Derinlik

#### 🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde Mikroservis Mimarisi: Servis Ayrıştırma & Servis Keşfi Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde Microservices mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): Discovery katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

### 🔍 MAANG Technical Deep-Dive

#### 🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Microservices Architecture: Decomposition & Service Discovery: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), Microservices implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in Discovery don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

### ☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```
// Java 25 & Spring Boot: Microservices & Discovery Kurumsal Uygulaması
package com.maang.se.module21.lesson08;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class MicroservicesArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record MicroservicesRequest(String transactionId, String payload, Instant timestamp) {
        public MicroservicesRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record MicroservicesResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class MicroservicesExecutionService {
        public MicroservicesResult processRequest(MicroservicesRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new MicroservicesResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering Microservices Engine Başlatıldı...");
        var service = new MicroservicesExecutionService();
        var req = new MicroservicesRequest("TXN-" + System.currentTimeMillis(), "Valid Data",
Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

### ⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

**🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN**

Microservices kurgulanırken Discovery bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

**🇬🇧 INTERVIEW TRAP & ANTI-PATTERN**

Coupling Microservices logic directly to synchronous blocking calls into Discovery is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

### 🎯 Mülakatta Hatırlanacak Altın Kurallar

• Microservices mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

### 🎯 Key Architectural Takeaways

• Design Microservices components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

209

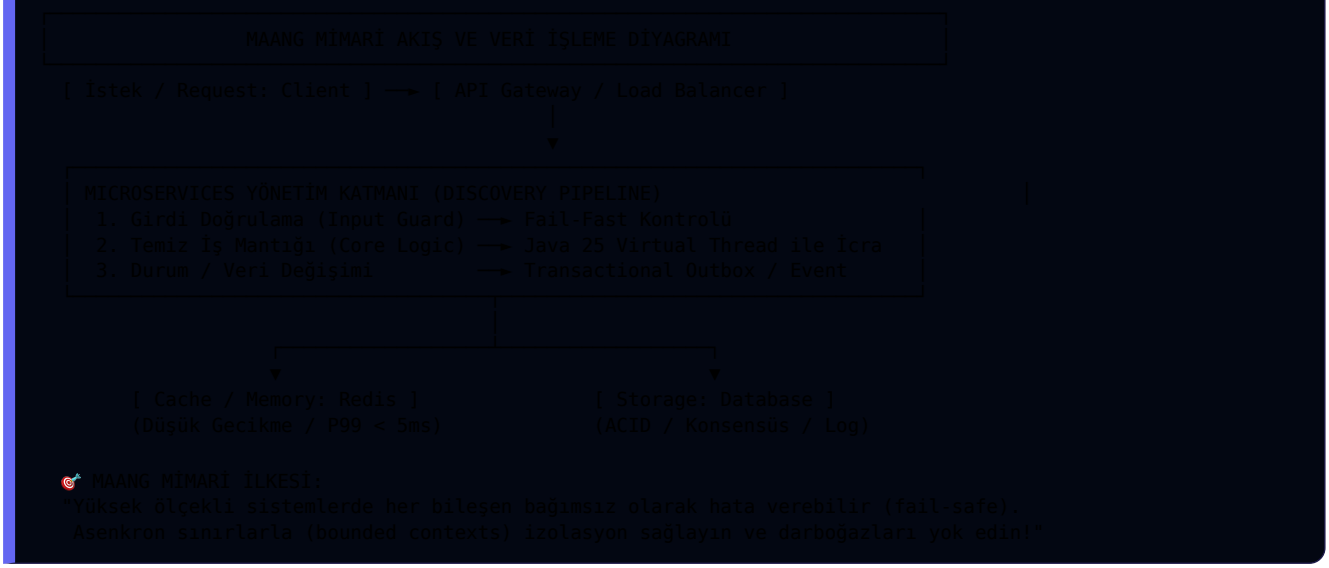
BÖLÜM: MIKROSERVIS MIMARISI: SERVIS AYRIŞTIRMA & SERVIS KEŞFI | MODULE: MICROSERVICES ARCHITECTURE: DECOMPOSITION & SERVICE DISCOVERY

# Mikroservis Mimarisi: Servis Ayrıştırma & Servis Keşfi - Gözlemlenebilirlik, Metrikler, İzleme ve Canlı Ortam Hata Ayıklama

Microservices Architecture: Decomposition & Service Discovery - Observability, Telemetry, Tracing & Production Troubleshooting

## ⚡ Mimari Akış ve Süreç Şeması

## ⚡ Process & Architecture Schematic




## 💡 Aptala Anlatım (ELI5 Metaforu)

## 💡 ELI5 Real-Life Metaphor

### 🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (Microservices mikro-servisleri) ve otomatik yönlendirme robotları (Discovery akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

### 🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (Microservices microservices), and dynamic routing switches (Discovery pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

### 🔍 MAANG Seviyesi Teknik Derinlik

#### 🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde Mikroservis Mimarisi: Servis Ayrıştırma & Servis Keşfi Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde Microservices mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): Discovery katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

### 🔍 MAANG Technical Deep-Dive

#### 🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Microservices Architecture: Decomposition & Service Discovery: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), Microservices implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in Discovery don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

### ☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```
// Java 25 & Spring Boot: Microservices & Discovery Kurumsal Uygulaması
package com.maang.se.module21.lesson09;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class MicroservicesArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record MicroservicesRequest(String transactionId, String payload, Instant timestamp) {
        public MicroservicesRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record MicroservicesResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class MicroservicesExecutionService {
        public MicroservicesResult processRequest(MicroservicesRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new MicroservicesResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering Microservices Engine Başlatıldı...");
        var service = new MicroservicesExecutionService();
        var req = new MicroservicesRequest("TXN-" + System.currentTimeMillis(), "Valid Data",
Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

### ⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

**🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN**

Microservices kurgulanırken Discovery bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

**🇬🇧 INTERVIEW TRAP & ANTI-PATTERN**

Coupling Microservices logic directly to synchronous blocking calls into Discovery is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

### 🎯 Mülakatta Hatırlanacak Altın Kurallar

• Microservices mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

### 🎯 Key Architectural Takeaways

• Design Microservices components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

210

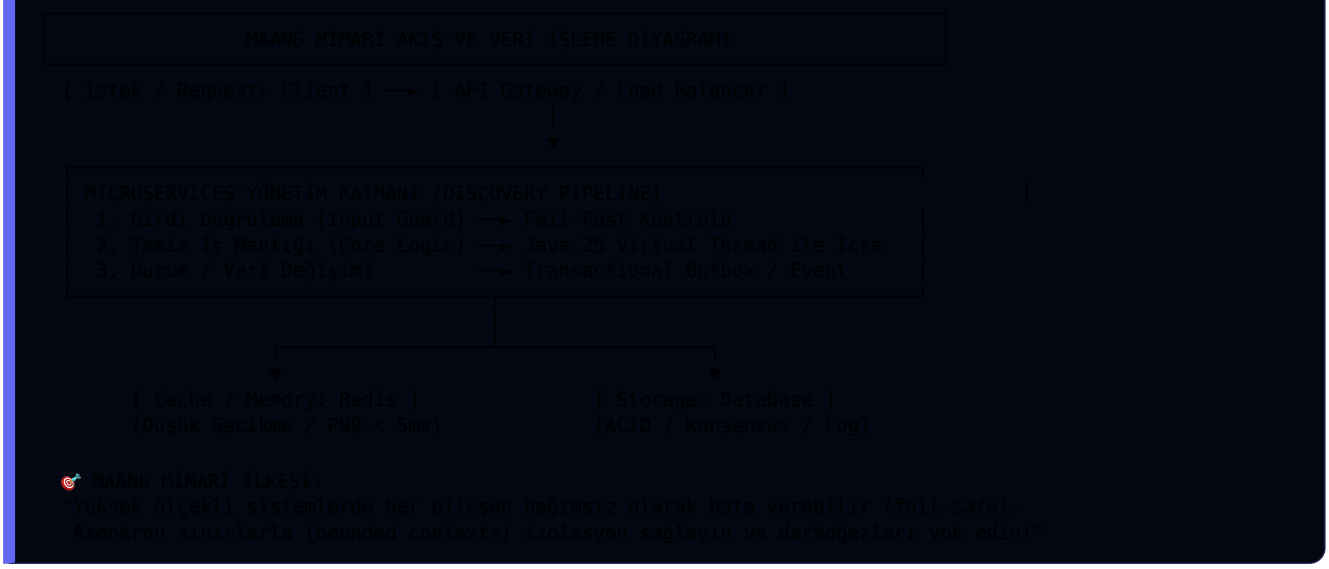
BÖLÜM: MIKROSERVIS MIMARISI: SERVIS AYRIŞTIRMA & SERVIS KEŞFI | MODULE: MICROSERVICES ARCHITECTURE: DECOMPOSITION & SERVICE DISCOVERY

# Mikroservis Mimarisi: Servis Ayrıştırma & Servis Keşfi - MAANG Mülakat Ustalığı, Mimari Ödünleşimler (Trade-Offs) & Staff+ Vizyonu

Microservices Architecture: Decomposition & Service Discovery - MAANG Interview Mastery, Architectural Trade-Offs & Staff+ Vizyonu

## ⚡ Mimari Akış ve Süreç Şeması

## ⚡ Process & Architecture Schematic




## 💡 Aptala Anlatım (ELI5 Metaforu)

## 💡 ELI5 Real-Life Metaphor

### 🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (Microservices mikro-servisleri) ve otomatik yönlendirme robotları (Discovery akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

### 🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (Microservices microservices), and dynamic routing switches (Discovery pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

### 🔍 MAANG Seviyesi Teknik Derinlik

#### 🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde Mikroservis Mimarisi: Servis Ayrıştırma & Servis Keşfi Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde Microservices mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): Discovery katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

### 🔍 MAANG Technical Deep-Dive

#### 🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Microservices Architecture: Decomposition & Service Discovery: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), Microservices implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in Discovery don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

### ☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```
// Java 25 & Spring Boot: Microservices & Discovery Kurumsal Uygulaması
package com.maang.se.module21.lesson10;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class MicroservicesArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record MicroservicesRequest(String transactionId, String payload, Instant timestamp) {
        public MicroservicesRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record MicroservicesResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class MicroservicesExecutionService {
        public MicroservicesResult processRequest(MicroservicesRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new MicroservicesResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering Microservices Engine Başlatıldı...");
        var service = new MicroservicesExecutionService();
        var req = new MicroservicesRequest("TXN-" + System.currentTimeMillis(), "Valid Data",
Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

### ⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

**🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN**

Microservices kurgulanırken Discovery bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

**🇬🇧 INTERVIEW TRAP & ANTI-PATTERN**

Coupling Microservices logic directly to synchronous blocking calls into Discovery is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

### 🎯 Mülakatta Hatırlanacak Altın Kurallar

• Microservices mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

### 🎯 Key Architectural Takeaways

• Design Microservices components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.